



Patrons de Conception

Akil ELKAMEL

akil.elkamel@isimm.u-monastir.tn

Supports de cours / Exercices / TDs



<https://github.com/akilelkamel/design-patterns>

Plan du cours

- Introduction et bases de la conception et programmation orientées objet
- Introduction aux patrons de conceptions
- Les patrons de création
- Les patrons de structure
- Les patrons de comportement

Chapitre 1:

Introduction et bases de la conception et programmation orientées objet

Pourquoi faire des efforts de conception?

- Il faut avoir en tête la seule constante du développement:
Le changement !
- De nombreux événements conduiront à modifier le code d'une application :
 - Modification des fonctionnalités et modèles
 - Augmentation du nombre d'utilisateurs et des volumes de données (parallélisation, etc.)
 - Obsolescence des outils (bibliothèques, frameworks, services tiers, etc.)
 - Changement de cible (desktop, serveur, mobile, cloud, etc.)
 - Choix technologique et/ou politique (base de données, OpenSource, etc.)
- Sans effort de conception, plus le temps passera, plus il sera difficile, coûteux et risqué de traiter ces changements.

Quels sont les critères qualités d'un programme?

- Il conviendra de **cibler plusieurs critères qui guideront la conception** :
 - **Fiabilité** : Pouvoir faire tourner le programme de manière fiable, sans risque d'erreur d'exécution.
 - **Maintenabilité** : Pouvoir facilement corriger un bug, mettre à jour les dépendances, etc.
 - **Performance** : Minimiser le temps d'exécution, la consommation de RAM, de stockage, etc.
 - **Évolutivité** : Pouvoir ajouter facilement des fonctionnalités sans risque de régression.
 - **Réutilisabilité** : Pouvoir réutiliser un code dans un autre contexte.
 - **Portabilité** : Pouvoir porter facilement un programme développé pour une cible vers une autre.
 - **Testabilité** : Pouvoir écrire facilement des tests unitaires et fonctionnels pour un programme.
 - Etc ...

La programmation orientée objet (POO)

- Un paradigme omniprésent
 - De nombreux programmeurs manipulent des objets sans forcément en avoir conscience :
 - En appelant la méthode `toUpperCase()` sur une chaîne de caractères `str`, par exemple `str.toUpperCase()`, nous faisons appel à la notation pointée typique de la P.O.O.
 - En Java, les chaînes de caractères sont représentées par des objets de la classe `String`.

```
Personne p1 = new Personne(...);
```

```
System.out.println("Nom: " + p1.getName());
```

Classe?

Objet?

La programmation orientée objet (POO)

Un paradigme difficile à maîtriser

- Utiliser des classes existantes ne demandera pas d'effort.
- Il sera plus délicat de :
 - Concevoir proprement ses propres classes
 - Faire un choix entre la définition d'une classe ou le recours à un autre paradigme (ex : définition d'une fonction) quand le langage le permet.

Les concepts de base de la P00

- Les classes avec leurs attributs et leurs méthodes
- Les constructeurs
- Les objets
- La visibilité (public, private, protected)
- Les accesseurs (get, set)
- Les relations (association, composition, agrégation)
- L'héritage et le polymorphisme
- Les méthodes et classes abstraites
- Les interfaces
- Les méthodes statiques
- Les attributs statiques

Les principes de la conception/programmation OO

- Abstraction:
 - Masquer les détails complexes et ne montrer que les éléments essentiels.
- Encapsulation:
 - Regrouper les données et les méthodes dans une classe et contrôler leur accès.
- Héritage:
 - Permettre à une classe de réutiliser et d'étendre les propriétés et méthodes d'une autre classe.
- Polymorphisme:
 - Permettre à un même appel de méthode d'avoir des comportements différents selon l'objet concerné.

Les principes de la conception/programmation OO

- Les **5 principes fondamentaux** regroupés dans l'acronyme **SOLID** :
 - **Single Responsibility Principle (SRP, responsabilité unique)**
 - **Open Closed Principle (OCP, ouvert/fermé)**
 - **Liskov Substitution Principle (LSP, substitution de Liskov)**
 - **Interface Segregation Principle (ISP, ségrégation des interfaces)**
 - **Dependency Inversion Principle (DIP, inversion des dépendances)**

SRP: Single Responsibility Principle

- Principe
 - Une classe doit avoir une seule responsabilité et une seule raison de changer.
- Autrement dit :
 - Une classe doit se concentrer sur une seule tâche principale.
 - Eviter de regrouper dans une même classe des responsabilités différentes.
 - Chaque objet doit avoir une responsabilité unique, et tous ses services doivent être alignés sur cette responsabilité.
 - La « responsabilité » se définit comme « une raison de changer ».
 - Cela facilite la maintenance, les tests et l'évolution du code.

Exemple : problème

```
class Employee {  
    private String name;  
  
    public void calculateSalary() {  
        // Calcul du salaire  
    }  
  
    public void saveToDatabase() {  
        // Sauvegarde dans la base de données  
    }  
  
    public void generateReport() {  
        // Génération d'un rapport  
    }  
}
```

Exemple : problème

- La classe **Employee** possède plusieurs responsabilités:
 - gérer les informations de l'employé
 - calculer son salaire
 - sauvegarder dans la base de données
 - générer un rapport

Exemple : solution

```
class Employee {
    private String name;
    public void calculateSalary() {
        // Calcul du salaire
    }
}

class EmployeeRepository {
    public void save(Employee employee) {
        // Sauvegarde en base
    }
}

class EmployeeReport {
    public void generate(Employee employee) {
        // Génération du rapport
    }
}
```

OCP: Open/Closed Principle

- Principe
 - Une classe doit être ouverte à l'extension, mais fermée à la modification.
- Cela signifie:
 - pouvoir ajouter de nouveaux comportements
 - sans modifier le code existant
 - en utilisant notamment l'héritage et les interfaces

Exemple : problème

```
class Payment {  
    public void pay(String type) {  
        if (type.equals("CARD")) {  
            // Paiement par carte  
        }  
        else if (type.equals("PAYPAL")) {  
            // Paiement PayPal  
        }  
    }  
}
```

Problème?

Pour ajouter un nouveau moyen de paiement, il faut **modifier la classe Payment**.

Exemple : solution

```
interface PaymentMethod {  
    void pay();  
}
```

```
class CardPayment implements PaymentMethod {  
    public void pay() {  
        System.out.println("Paielement par carte");  
    }  
}
```

```
class PayPalPayment implements PaymentMethod {  
    public void pay() {  
        System.out.println("Paielement PayPal");  
    }  
}
```

Exemple : solution

- On peut maintenant ajouter :

```
class CashPayment implements PaymentMethod {  
    public void pay() {  
        System.out.println("Paielement en espèces");  
    }  
}
```

LSP: Liskov Substitution Principle

- Principe
 - Un objet d'une classe fille doit pouvoir remplacer un objet de sa classe mère sans modifier le comportement attendu du programme.
- Autrement dit :
 - Une sous-classe doit respecter le contrat de sa classe mère.
 - L'héritage doit représenter une véritable relation « est un ».

Exemple : problème

```
class Bird {  
    public void fly() {  
        System.out.println("Je vole");  
    }  
}
```

```
class Penguin extends Bird {  
    @Override  
    public void fly() {  
        throw new UnsupportedOperationException();  
    }  
}
```

Exemple : problème

- Problème: un **Penguin** est un **Bird**, mais il ne peut pas voler.

- Le remplacement :

```
Bird bird = new Penguin();  
bird.fly();    // Erreur !
```

viole le principe de substitution.

Solution: On sépare les comportements

```
class Bird {
    public void eat() {
        System.out.println("Je mange");
    }
}

interface Flyable {
    void fly();
}

class Eagle extends Bird implements Flyable {
    public void fly() {
        System.out.println("Je vole");
    }
}

class Penguin extends Bird {
    // Pas de méthode fly()
}
```

Penguin peut être utilisé
comme un **Bird** sans provoquer
de comportement inattendu.

ISP: Interface Segregation Principle

- Principe
 - Une classe ne doit pas être obligée de dépendre de méthodes qu'elle n'utilise pas.
- Donc :
 - préférer plusieurs petites interfaces spécialisées
 - plutôt qu'une seule interface contenant trop de méthodes.

Exemple : problème

```
interface Worker {  
    void work();  
    void eat();  
}  
  
class Robot implements Worker {  
    public void work() {  
        System.out.println("Le robot travaille");  
    }  
  
    public void eat() {  
        // Le robot ne mange pas !  
    }  
}
```

Robot est obligé d'implémenter **eat()** alors qu'il n'en a pas besoin.

Exemple : solution

```
interface Workable {  
    void work();  
}
```

```
interface Eatable {  
    void eat();  
}
```

```
class Human implements Workable, Eatable {  
    public void work() {  
        System.out.println("L'humain travaille");  
    }  
  
    public void eat() {  
        System.out.println("L'humain mange");  
    }  
}
```

Exemple : solution

```
class Robot implements Workable {  
    public void work() {  
        System.out.println("Le robot travaille");  
    }  
}
```

Chaque classe dépend uniquement des méthodes **dont elle a besoin.**

DIP: Dependency Inversion Principle

- Principe
 - Les modules de haut niveau ne doivent pas dépendre directement des modules de bas niveau. Les deux doivent dépendre d'abstractions.
- En pratique:
 - dépendre d'une interface plutôt que d'une classe concrète
 - réduire le couplage entre les composants
 - faciliter le remplacement et les tests.

Exemple : problème

```
class MySQLDatabase {  
    public void save(String data) {  
        System.out.println("Sauvegarde MySQL");  
    }  
}  
  
class UserService {  
    private MySQLDatabase database = new MySQLDatabase();  
  
    public void saveUser(String user) {  
        database.save(user);  
    }  
}
```

UserService dépend directement de **MySQLDatabase**.
Si on veut passer à PostgreSQL, il faut modifier **UserService**.

Exemple : solution

```
interface Database {  
    void save(String data);  
}
```

```
class MySQLDatabase implements Database {  
    public void save(String data) {  
        System.out.println("Sauvegarde MySQL");  
    }  
}
```

```
class PostgreSQLDatabase implements Database {  
    public void save(String data) {  
        System.out.println("Sauvegarde PostgreSQL");  
    }  
}
```

Exemple : solution

```
class UserService {  
    private Database database;  
  
    public UserService(Database database) {  
        this.database = database;  
    }  
  
    public void saveUser(String user) {  
        database.save(user);  
    }  
}
```

- Utilisation :

```
UserService s = new UserService(new MySQLDatabase());
```

ou :

```
UserService s = new UserService(new PostgreSQLDatabase());
```

**UserService dépend de l'abstraction Database,
et non d'une implémentation particulière.**

Principes SOLID: synthèse

Principe	Idée essentielle
S - SRP	Une classe → une responsabilité
O - OCP	Etendre sans modifier
L - LSP	Une sous-classe doit pouvoir remplacer sa classe mère
I - ISP	Plusieurs petites interfaces plutôt qu'une grosse
D - DIP	Dépendre des abstractions, pas des implémentations